

Cahier de TP

«Design Pattern pour les micro-services»

Version Quarkus

Outils utilisés :

- Bonne connexion Internet
- Système d'exploitation recommandé : Linux, MacOS, Windows 10
- JDK21+
- IDE de votre choix
- Git
- Docker
- JMeter

Dépôt des solutions :

<http://github.com/dthibau//dp-microservices-solutions-quarkus.git>

Table des matières

Atelier 1: Décomposition.....	3
Atelier 2: Interaction synchrone, Discovery et Circuit Breaker.....	6
2.1 Préparation environnement kind.....	6
2.2 Scaling de product-service.....	6
2.3 Interaction synchrone et Load-balancing.....	7
2.4 Circuit Breaker Pattern.....	7
Atelier 3: Interaction asynchrone.....	8
3.1 Démarrage du Message Broker.....	8
3.2 Mise en place du producer.....	8
3.3 Mise en place du consommateur.....	9
3.4 Messagerie transactionnelle.....	9
Atelier 4: Saga Pattern.....	10
Atelier 5: Logique métier.....	11
5.1 Domain Model et Domain Event Pattern.....	11
Atelier 6: Service de Query.....	12
6.1 API Composition.....	12
6.2 CQRS View Pattern.....	12
Atelier 7: API Gateway.....	13
Atelier 8: Tests.....	14
8.1 Test d'intégration.....	14
8.2 Design By Contract et Test de composants.....	14
Atelier 9: Observabilité.....	15
9.1 Health & métriques (équivalent Actuator).....	15
9.2 Métriques CircuitBreaker.....	15
9.3 Tracing avec OpenTelemetry et Zipkin.....	15

Atelier 10: Kubernetes.....	17
10.1 Construction d'image.....	17
10.2 Ressources Kubernetes.....	17
10.3 Configuration externalisée.....	18
10.4 : Déploiements de la stack.....	18
10.3.1 Kafka.....	18
10.3.2 Gateway.....	18
10.3.3 Services applicatifs.....	19
10.4 : Maillage de service, Istio.....	19
10.5 Dashboard Istio/Kiali.....	19

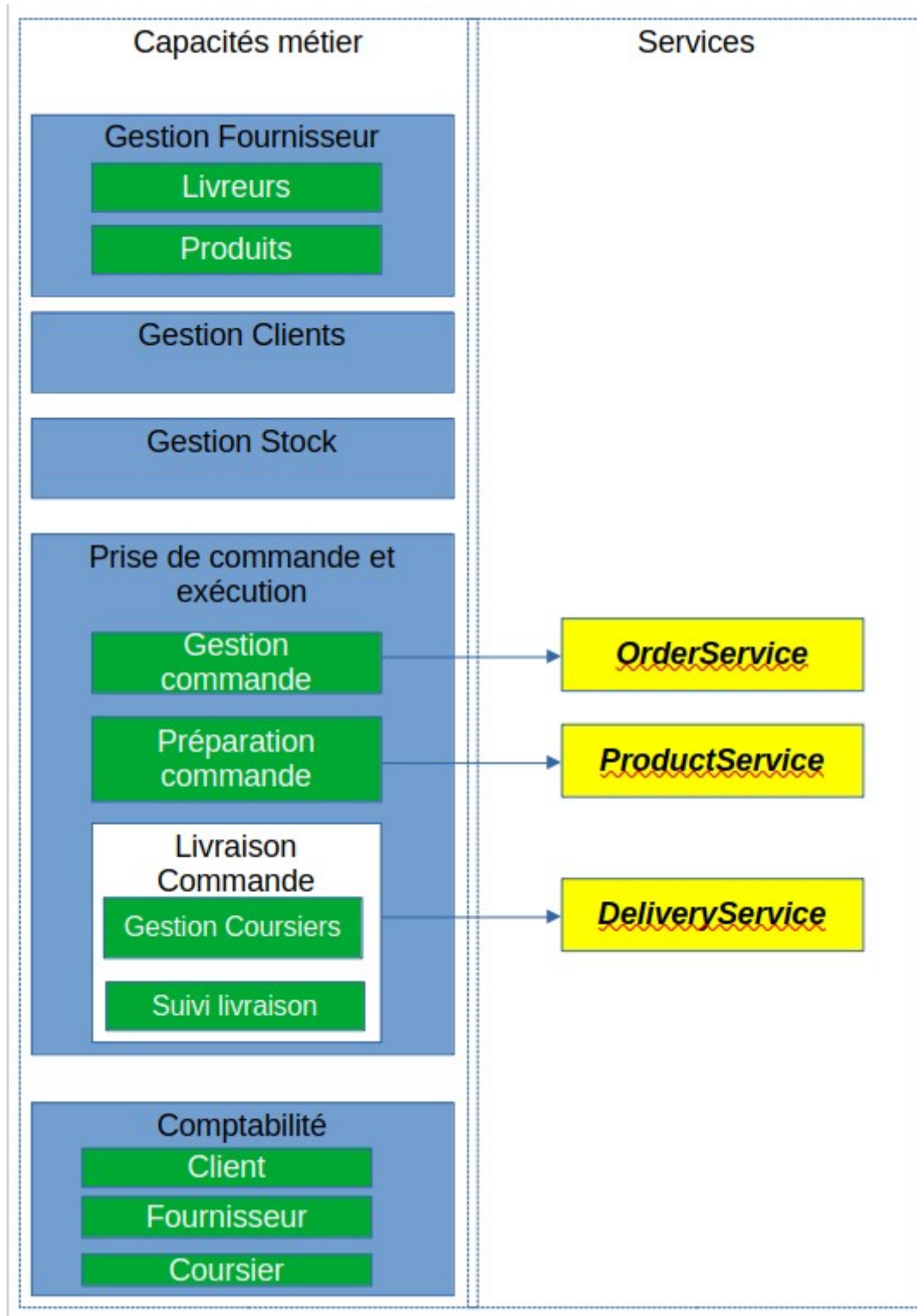
Atelier 1: Décomposition

Objectifs : Illustrer le pattern « Decompose by business capability »

Notre système représente un magasin permettant de commander des produits en ligne.

Les produits sont ensuite déstockés et livrés au domicile du client.

Les capacités métier de notre magasin



Nous nous intéressons principalement à la prise et l'exécution de la commande qui a donc identifié 3 micro-services

- **OrderService** : Création, Mise à jour, historique de commande
- **ProductService** : Récupération des produits, emballage
- **DeliveryService** : Disponibilité des coursiers, suivi de livraison

Les APIs ont été déduites de la Story « Passer une commande »

Les opérations identifiées et leurs collaborateurs pour ce UserStory sont résumés dans le tableau suivant :

Service	Opération	Collaborateur
OrderService	<i>createOrder()</i> <i>findOrderDetail()</i>	ProductService <i>createTicket()</i> AccountingService <i>executePayment()</i>
ProductService	<i>createTicket()</i> <i>noteTicketReadyToPickup()</i>	DeliveryService <i>createDelivery()</i>
DeliveryService	<i>noteDeliveryPickUp()</i> <i>updatePosition()</i> <i>noteDeliveryDelivered()</i> <i>findAvailableCourier()</i> <i>findPendingDeliveries()</i>	

Commandes

OrderService

- *createOrder()* :
 - Entrée : consumerId, orderLineItems, paymentInfo, deliveryInfo
 - Retour : orderId
 - Post-conditions : Paiement effectué

ProductService

- *createTicket()*
 - Entrée : orderId, orderLineItems
 - Retour : ticketId
 - Pré-conditions : Les produits demandés sont disponibles en stock
- *noteTicketReadyToPickup()*
 - Entrée : ticketId
 - Retour : Void
 - Post-conditions : La livraison correspondante est créée

DeliveryService

- *createDelivery()*
 - Entrée : ticketId, orderId
 - Retour : deliveryId
 - Post-conditions: Création d'une livraison dans l'état TOPICK_UP
- *noteDeliveryPickUp()*
 - Entrée : deliveryId, courierId
 - Retour : Void
 - Post-conditions : Livraison dans l'état IN_PROGRESS, coursier affecté
- *updatePosition()*
 - Entrée : courierId

- Retour : Void
- Post-conditions : Mise à jour position coursier
- `noteDeliveryDelivered()`
 - Entrée : `deliveryId`,
 - Retour : Void
 - Post-conditions : Livraison dans l'état DELIVERED, Coursier disponible

Requêtes

order-service

- `findOrderDetail(orderId)` : Toutes les informations d'une commande

product-service

- `findTicket(orderId)` : Le ticket associé à une commande

delivery-service

- `findAvailableCouriers()` : Les coursiers disponibles
- `findDeliveries(deliveryStatus)` : Les livraisons en fonction de leurs statuts
- `findDelivery(orderId)` => La livraison associée à la commande

Récupérer le tag **Atelier1** des solutions

Récupérer les projets fournis et les importer dans vos IDE.

Démarrer chacun des services et via la DevServices Quarkus :

- accéder à la documentation Swagger
- Visualiser les tables de la base H2

Atelier 2: Interaction synchrone, Discovery et Circuit Breaker

Objectifs : Comprendre les pré-requis à une interaction synchrone dans un contexte de réplication de services et de résilience

Dans un environnement Quarkus, le service de discovery est fourni par un Kubernetes, le service de configuration centralisée peut être fourni par l'extension

2.1 Préparation environnement kind

Installation de *kind*

Démarrage d'un cluster avec une config Ingress :

```
kind create cluster --config kind.yml
```

Installer *Ingress*

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml
kubectl wait -n ingress-nginx --for=condition=Available deploy/ingress-nginx-controller --timeout=180s
```

2.2 Scaling de product-service

Dans le projet *product-service*, ajouter les dépendances suivantes :

```
quarkus ext add \
  quarkus-smallrye-health \
  quarkus-smallrye-fault-tolerance \
  quarkus-kubernetes \
  quarkus-kubernetes-config \
  quarkus-container-image-jib \
  quarkus-kind
```

Générer l'image de product-service

```
./mvnw package -DskipTests
```

Charger l'image dans kind

```
kind load docker-image plb/product-service:1.0.0 --name=plb-dev
```

Déployer les ressources kubernetes

```
quarkus build
```

Scaler product-service

```
kubectl scale deployment product-service --replicas=3
```

Vérifier les 3 pods qui s'exécutent

Déployer une ressource Ingress pour y accéder :

```
kubectl apply -f src/main/resources/ingress-product.yml
```

Vérifier la répartition de charge en accédant à :

2.3 Interaction synchrone et Load-balancing

Vérifier l'extension

quarkus-rest-client-jackson

Visualiser le code de OrderService et en particulier le package service

Utiliser le script JMeter *TPS/2.2/CreateOrder.jmx* fourni pour visualiser la répartition de charge

Arrêter une le déploiement de product-service et observer le comportement de OrderService

```
kubectl delete deployment product-service
```

2.4 Circuit Breaker Pattern

Dans le projet OrderService, ajouter l'extension *quarkus-smallrye-fault-tolerance*

Encapsuler le code de l'appel du service ProductService dans un circuit breaker.

Atelier 3: Interaction asynchrone

Objectifs : Comprendre les pré-requis aux interactions asynchrones, le découplage producteur / consommateur et le rôle du Transactional Outbox Pattern

Description

Nous mettons en place une communication de type Publish/Subscribe

ProductService publie l'événement TICKET_READY vers le topic tickets DeliveryService réagit en créant une livraison

3.1 Démarrage du Message Broker

3.2 Mise en place du producer

Ajouter la dépendance *quarkus-messaging* et *quarkus-messaging-kafka* sur product-service

Visualiser la classe org.formation.service.TicketService comportant 2 méthodes

- `public Ticket createTicket(Long orderId, List<ProductRequest> productsRequest)`

Crée un ticket avec le statut TicketStatus.CREATED

- `public Ticket readyToPickUp(Long ticketId)`

Modifie le statut du ticket en TicketStatus.READY_TO_PICK

Publie un événement READY_TO_PICK sur le topic Kafka tickets, le corps du message est une instance de la classe ChangeStatusEvent

L'annotation **@Transactional** sur les classes services englobe les méthodes dans une transaction BD.

Cependant si une erreur survient lors de l'envoi du message la transaction n'est pas rollbackée.

Pour générer un message, commencer par créer un ticket via swagger

POST /api/tickets/1

```
[
  {
    "reference": "REF",
    "quantity": 2
  }
]
```

Puis provoquer l'envoi du message via

POST /api/tickets/1/pickup

Visualisez le topic créé et le message envoyé via la console d'administration Kafka

3.3 Mise en place du consommateur

Les mêmes dépendances ont été ajoutées sur *delivery-service*

Visualiser la classe *org.formation.service.DeliveryService* qui écoute les messages du topic tickets

Vous pouvez tester le tout avec le script JMeter fourni

Lors de l'exécution du test, vous pouvez arrêter *delivery-service* puis le redémarrer tous les messages sont consommés

3.4 Messagerie transactionnelle

Implémenter une messagerie transactionnelle via le pattern Transaction Outbox

Atelier 4: Saga Pattern

Objectifs : Implémenter le pattern Saga via un orchestrateur

Nous voulons implémenter la saga suivante :

Étape	Service	Transaction	Transaction de compensation
1	OrderService	<i>createOrder()</i>	<i>rejectOrder()</i>
2	ProductService	<i>createTicket()</i>	<i>rejectTicket()</i>
3	PaymentService	<i>authorizePayment()</i>	
4	ProductService	<i>approveTicket()</i>	
5	OrderService	<i>approveOrder()</i>	

Nous implémentons le pattern via un orchestrateur **CreateOrderSaga** de OrderService.

L'orchestrateur envoie des messages commande dans le style request/response :

1. TicketCommande/TICKET_CREATE , sur le channel ***tickets-command***

Il attend une réponse sur le channel ***order-response***

2. A la réception d'une réponse à TICKET_CREATE:

- Si OK : Envoi d'une commande de paiement sur le channel ***payments-command***
- Si NOK : Transaction locale compensatoire : la commande est annulée

3. A la réception d'une réponse à PAYMENT_AUTHORIZE:

- Si OK :
 - Envoi d'une commande TICKET_APPROVE via le channel ***tickets-command***
 - Transaction locale pour valider la commande
- Si NOK :
 - Envoi d'une commande TICKET_REJECT via le channel ***tickets-command***
 - Transaction locale compensatoire : la commande est annulée

Les services impliqués ***product-service*** et ***payment-service*** implémentent les réponses aux requêtes de l'orchestrateur.

Visualisez le code de la solution

Vous pouvez tester soit via l'interface Swagger de order-service, soit en utilisant le fichier JMeter TPS/2.2/CreateOrder.jmx

paymlent-service refuse la transaction si le *paymentToken* ne commence pas par le caractère « A »

Atelier 5: Logique métier

5.1 Domain Model et Domain Event Pattern

L'objectif est d'appliquer ces 2 patterns au service **product-service** (qui pour l'instant implémente un *TransactionScriptPattern* ou la classe *org.formation.service.TicketService* contient la logique métier).

Définir une classe **ResultDomain** encapsulant :

- Un agrégat *Ticket*
- Un événement : *TicketStatusEvent*

La logique métier relatif au Ticket est implémentée dans la classe Ticket. Celle ceci expose :

- *public static ResultDomain createTicket(Long orderId, List<ProductRequest> productRequest) throws MaxWeightExceededException*
- *public ResultDomain approveTicket()*
- *public ResultDomain rejectTicket()*
- *public ResultDomain readyToPickUp()*

Le code de classe service est donc de déléguer la logique métier à la classe Ticket, récupérer le *ResultDomain* résultant et systématiquement :

- stocker l'objet du domaine en base
- publier l'événement sur la channel **ticket-event** (Transactional Outbox Pattern)

Visualiser le code

Vous pouvez tester soit en utilisant le fichier JMeter TPS/2.2/CreateOrder.jmx

Atelier 6: Service de Query

Vous pouvez utiliser le script JMeter TPS/6/CreateAndDeliverOrder.jmx fourni pour initialiser les bases avec des données (Order, Ticket, Livraison)

6.1 API Composition

Nous voulons pouvoir visualiser toutes les informations d'un livreur affecté à une commande.

Créer un nouveau micro-service ***order-query-service*** qui applique le pattern API Composition pour implémenter cette fonctionnalité.

Ensuite, implémenter un point d'accès qui affiche toutes les commandes en cours

6.2 CQRS View Pattern

Implémenter le pattern CQRS, le service s'abonne aux événements métier *OrderEvent* et *DeliveryEvent* pour mettre à jour une table locale stockant la jointure entre Order et Livraison

Atelier 7: API Gateway

Création d'un nouveau service Gateway qui route les URLs externes vers :

- Le endpoint permettant de créer une Commande
- Le endpoint permettant d'indiquer qu'un ticket est prêt
- Le endpoint permettant au livreur de prendre une Livraison
- Le endpoint permettant de faire des requêtes

Atelier 8: Tests

8.1 Test d'intégration

Reprendre le tag Atelier8.1

Visualiser la classe de test LivraisonRepositoryTest

L'exécuter et regarder les beans présents lors du test

8.2 Design By Contract et Test de composants

Atelier 9: Observabilité

9.1 Health & métriques (équivalent Actuator)

Extensions à ajouter (dans **chaque** microservice)

```
./mvnw quarkus:add-extension -Dextensions="quarkus-smallrye-health,quarkus-micrometer-registry-prometheus"
```

Endpoints exposés

- **Health :**
 - GET /q/health (global)
 - GET /q/health/live, GET /q/health/ready, GET /q/health/started
- **Metrics (Prometheus) :**
 - GET /q/metrics

HealthCheck Custom :

```
@Readiness
@ApplicationScoped
public class DbReadinessCheck implements HealthCheck {
    @Override public HealthCheckResponse call() {
        return HealthCheckResponse.up("db");
    }
}
```

9.2 Métriques CircuitBreaker

Les annotations **SmallRye Fault Tolerance** publient des métriques (compteurs, états) exposées sur **/q/metrics** au format Prometheus (ex. `ft_circuitbreaker_open_total`, `ft_retry_calls_total`, etc.).

Récupérer le répertoire grafana fourni et y exécuter :

```
docker-compose up -d
```

Cela doit démarrer un serveur Prometheus et Grafana

Se connecter sur Grafana à

`http://localhost:3000`

et se connecter avec `admin/admin`

La source de donnée Prometheus et le Dashboard Grafana sont déjà configurés dans les containers.

9.3 Tracing avec OpenTelemetry et Zipkin

Avec Quarkus on utilise **OpenTelemetry** + un exporter **Zipkin**.

```
./mvnw quarkus:add-extension -Dextensions="quarkus-opentelemetry,quarkus-opentelemetry-exporter-zipkin"
```

Démarrer un serveur Zipkin

```
docker run -d -p 9411:9411 -name zipkin openzipkin/zipkin
```

Accéder à <http://localhost:9411/zipkin/>

Ajouter pour tous les services la dépendance sur le starter zipkin et sleuth

Les redémarrer et visualiser la différence dans les traces

Ajouter pour chaque micro-services la configuration suivante :

```
# Activer OTel
```

```
quarkus.opentelemetry.enabled=true
```

```
# Échantillonnage 100%
```

```
quarkus.opentelemetry.tracer.sampler=parentbased_traceidratio
```

```
quarkus.opentelemetry.tracer.sampler.ratio=1.0
```

```
# Export Zipkin
```

```
quarkus.opentelemetry.tracer.exporter.zipkin.endpoint=http://localhost:9411/api/v2/spans
```

```
# (Optionnel) service name explicite
```

```
quarkus.application.name=order-service
```

```
quarkus.opentelemetry.tracer.resource-attributes=service.name=${quarkus.application.name}
```

Visualiser les traces dans Zipkin

Atelier 10: Kubernetes

Objectifs

- Construction d'image
- Déploiement vers un cluster
- Remote development / Debug development
- Variables d'environnement et ConfigMap

10.1 Construction d'image

Nécessite un compte DockerHub

Récupérer le projet delivery-service, il apporte les modifications suivantes :

- Mock du service notification-service durant les tests
- Définition d'une base H2 en profil prod

Ajouter l'extension **container-image-jib** et supprimer **container-image-docker**

Indiquer le nom de l'image dans *application.properties*, DockerHub ne vous laissera pousser que des images préfixées par votre compte

```
quarkus.container-image.image=dthibau/quarkus-delivery-service:1.0.0-SNAPSHOT
```

Construire l'image en activant le profil de prod:

```
quarkus build -Dquarkus.container-image.build=true -Dquarkus.profile=prod
```

Ensuite push vers DockerHub

```
docker login
docker push dthibau/quarkus-delivery-service:1.0.0-SNAPSHOT
```

Exécuter ensuite l'image dans l'environnement local et essayer d'accéder à l'application. Par exemple :

```
docker run --network host dthibau/quarkus-delivery-service:1.0.0-SNAPSHOT
```

10.2 Ressources Kubernetes

Démarrer votre cluster Kubernetes

Sur le projet delivery-service

Ajouter l'extension **kind**

```
quarkus ext add quarkus-kind
```

Effectuer un build et visualiser les ressources Kubernetes créées

```
quarkus build
```

Assurer vous que votre cluster Kubernetes est démarré et que votre client *kubectl* est effectif.
Déployer les ressources sur votre cluster avec kubectl :

```
kubectl apply -f target/kubernetes/kind.yml
```

Vérifier le démarrage du pod et la présence du service

Effectuer un port-forward afin de pouvoir accéder au service via un port local :

```
kubectl port-forward service/delivery-service 8001:80
```

10.3 Configuration externalisée

```
quarkus ext add quarkus-kubernetes-config
```

Propriétés de chaque service :

```
quarkus.kubernetes-config.enabled=true  
quarkus.kubernetes-config.config-maps=common-config,service-  
specific-config  
quarkus.kubernetes-config.secrets=global-secrets
```

10.4 : Déploiements de la stack

10.3.1 Kafka

Installation de Kafka via Helm
Installation de Helm

```
helm repo add bitnami https://charts.bitnami.com/bitnami  
helm install my-release bitnami/kafka
```

Exposition du service kafka sur l'adresse kafka:9092

```
kubectl apply -f k8s/kafka-micro.yml
```

Déploiement de la stack

Les images du TP précédent ont été poussées vers DockerHub

10.3.2 Gateway

Installation contrôleur Gateway dans Kube

```
kubectl apply -f https://raw.githubusercontent.com/nginxinc/nginx-gateway-fabric/main/deploy/single/standard/deploy.yaml
```

Déploiement d'une ressource Gateway et des routes pour la stack

```
kubectl apply -f gateway.yml
```

10.3.3 Services applicatifs

Utiliser les facilités de Quarkus pour déployer les services

Mettre au point un fichier deploy.sh et undeploy.sh qui déploie toute la stack

Utiliser le script JMeter fourni pour solliciter la stack

10.4 : Maillage de service, Istio

Installation Istio :

Voir <https://istio.io/docs/setup/getting-started/#download>

Dans un premier temps désactiver Istio

```
kubectl edit namespace default
```

Mettre au point un script

./deployment.sh pour déployer les micro-services sans la gateway

Vérifier le bon déploiement et le nombre de pods dans un contexte sans-istio

Activer istio dans le namespace par défaut

```
kubectl label namespace default istio-injection=enabled
```

Déployer la stack et regarder les pods

Déployer les ressources Istion :

- Une gateway Istio
- Un VirtualService définissant les règles de routage

```
kubectl apply -f istio.yaml
```

Vérifier le tout avec :

```
istioctl analyze
```

Accéder à des micro-services via la gateway :

```
export INGRESS_PORT=$(kubectl -n istio-system get service istio-ingressgateway -o jsonpath='{.spec.ports[?(@.name=="http2")].nodePort}')
export SECURE_INGRESS_PORT=$(kubectl -n istio-system get service istio-ingressgateway -o jsonpath='{.spec.ports[?(@.name=="https")].nodePort}')
export INGRESS_HOST=$(minikube ip)
```

Puis dans un navigateur :

[http://\\$INGRESS_HOST:\\$INGRESS_PORT/order-service](http://$INGRESS_HOST:$INGRESS_PORT/order-service)

10.5 Dashboard Istio/Kiali

Installer les add-ons istio : dans le répertoire d'istio :

```
kubectl apply -f samples/addons
```

Accès au tableau de bord kiali

```
istioctl dashboard kiali
```

